API Filtering, Sorting & Field Selection

flexible query parameters without table scans

Watch the 4-min video

In one line: give clients consistent filter, sort and fields parameters, but only on an **allow list** of fields that each have an index designed for real queries; reject anything else with a 400, use parameterized SQL, cap page sizes, and send fuzzy search to a search engine.

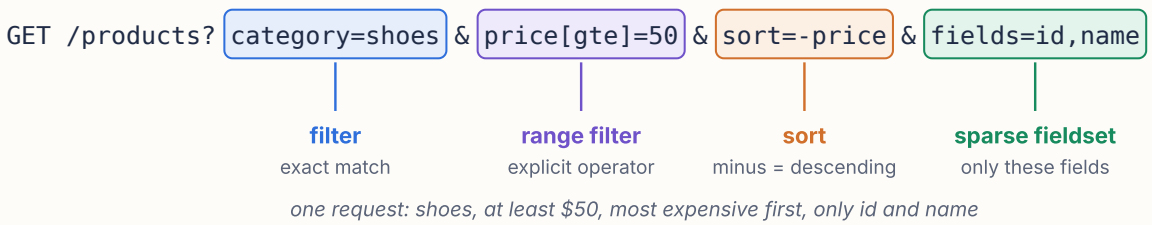
1 One request, three jobs

A shopper opens the shoes category, sorts by price from high to low, and the mobile app only needs the id and the name. One request expresses all three:

```
GET /products?category=shoes&sort=-price&fields=id,name
```

Designing these query parameters is easy. Designing them so they **don't melt your database** is the real interview question.

2 Consistent conventions



one request: shoes, at least $50, most expensive first, only id and name

Figure 1. Each kind of parameter has one predictable shape, so clients can guess the API without reading docs for every endpoint.

- **Filters** are plain query parameters named after fields: `category=shoes`, `status=active`.
- **Ranges** use explicit operators: `price[gte]=50`, `price[lt]=100` (gte = greater than or equal, lt = less than).
- **Sorting** uses one `sort` parameter, a comma-separated list where a minus sign means descending: `sort=-price,name`.
- **Field selection** uses `fields=id,name`, sometimes called **sparse fieldsets**.

Field selection is a cheap win. A mobile list needs 2 fields, not all 42. Smaller responses mean less bandwidth and faster parsing, and sometimes cheaper queries, if the server can skip joins for fields nobody asked for. The client chooses the shape without needing GraphQL.

3 The danger: filtering on anything

If **any** field can be filtered or sorted, someone will send `sort=description` or filter on a column with no index. On a table with **50 million rows**, that's a **full table scan** plus a big sort, and one slow query can saturate the database's CPU and I/O for every other user.

So never pass parameters straight into SQL:

- **Allow-list** the fields that can be filtered and the fields that can be sorted, and back each one with an index.
- **Reject unknown or unsupported parameters** with `400 Bad Request` and a helpful message, instead of silently ignoring them. A silently ignored typo like `categroy=shoes` returns every product and looks like a bug in your API.
- **Map public names to columns yourself**, so the API name `price` can point at `p.price_cents` and you can rename columns freely.
- **Always use parameterized queries** for values, to block SQL injection. Column names in `ORDER BY` can't be bound as parameters, which is one more reason they must come from your allow list.

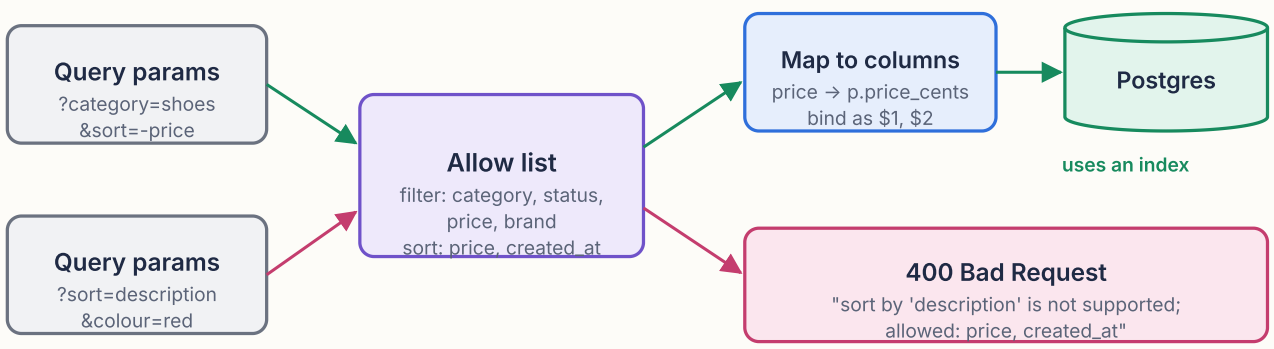


Figure 2. Allowed parameters are mapped to real columns and bound as parameters. Anything outside the list is rejected up front with a clear 400.

```
SORTABLE = {"price": "price_cents", "created_at": "created_at"}
FILTERABLE = {"category": "category", "status": "status", "brand": "brand"}

for name in query.sort:                 # e.g. ["-price"]
    col = SORTABLE.get(name.lstrip("-"))
    if col is None:
        raise HTTPException(400, f"cannot sort by {name}; allowed: {list(SORTABLE)}")
# values are never pasted into SQL:  WHERE category = $1 AND price_cents >= $2
```

4 Design indexes for the real access patterns

For *category + sort by price*, a **composite index on (category, price)** lets the database find the shoes already in price order and **stop after 20 rows**. Column order matters: the equality filter (category) comes first, the sort column (price) second. An index on price alone, or on (price, category), can't do this.



Figure 3. Inside the index all shoes sit together, ordered by price. The database seeks to the first one, reads 20 entries and stops, with no sort step and no table scan.

WORKED EXAMPLE

50M products, 2M of them shoes. `category=shoes&sort=-price&limit=20`:

- **No index:** scan 50M rows, keep 2M shoes, sort 2M rows by price, return 20. Seconds of CPU and I/O per request.
- **Index on (category) only:** find 2M shoes fast, but still sort 2M rows on every request.
- **Index on (category, price):** read ~20 entries in order. About a millisecond, and the same on page 1,000 with cursor pagination.

Add the tie-breaker for cursors: `(category, price, id)`.

Also: combine filters with **cursor pagination** so every page stays fast; **require a selective filter** on expensive endpoints (for example, orders must be filtered by customer or a date range); and **cap the page size** (default 20, maximum 100).

5 Filtering is not search

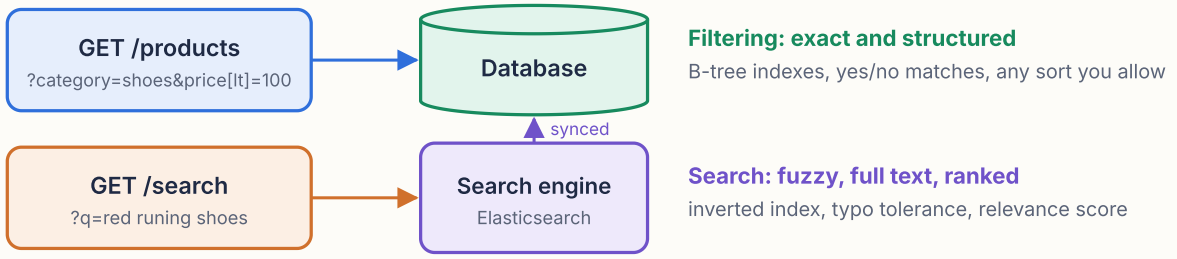


Figure 4. Exact, structured filters belong in the database with B-tree indexes. Fuzzy, ranked text search belongs in a search engine, usually behind its own endpoint.

Filtering is exact and structured: `category = shoes`, `price < 100`. A row matches or it doesn't. **Search** is fuzzy, full text and **ranked by relevance**: "red runing shoes" should still find red running shoes despite the typo, best match first. `LIKE '%red%'` can't use a normal index and has no ranking, so search belongs in a search engine such as **Elasticsearch** or **OpenSearch**, kept in sync from the database and exposed as a separate `/search` endpoint.

COMMON MISTAKES

- Accepting any column name for filter or sort: one request can trigger a 50M-row scan.
- Silently ignoring unknown parameters, so typos return everything.
- Building SQL by string concatenation from query parameters (SQL injection).
- Indexing each column separately when queries need one composite index in the right order.
- Implementing search with `LIKE '%term%'` on a big table.

6 Key terms

Filter: exact, structured condition on a field

Allow list: the only fields that may be filtered or sorted

Parameterized query: values bound separately from SQL text

Range operator: gte, gt, lte, lt in the parameter

Composite index: one index over several columns, in order

Search engine: inverted index, fuzzy, relevance-ranked

Sparse fieldset: client picks returned fields

Full table scan: reading every row to answer a query

Selective filter: condition that matches few rows

7 Interview questions

Q1. How do you prevent filter parameters from causing full table scans? BASIC
Allow-list filterable and sortable fields, back each with an index designed for real query patterns (e.g. (category, price)), reject unknown parameters with a 400, use parameterized queries, cap page size at ~100 and pair it with cursor pagination.

Q2. What's the difference between filtering and search? BASIC
Filtering is exact, structured matching in the database (category = shoes, price < 100) using B-tree indexes. Search is fuzzy, full-text, relevance-ranked matching with typo tolerance, served by a search engine like Elasticsearch or OpenSearch.

Q3. What are sparse fieldsets and why use them? BASIC
A fields parameter (fields=id,name) so clients get only what they need: 2 fields instead of 42 means smaller payloads, faster parsing, and sometimes skipped joins.

Q4. category=shoes&sort=-price is slow on 50M rows. Which index? SCENARIO
A composite index on (category, price), equality column first, sort column second, plus id as a tie-breaker for cursors. The database seeks to shoes and reads 20 entries in order with no sort step.

Q5. A client sends sort=description. What should the API do? SCENARIO
Return 400 with a message listing the allowed sort fields. Don't silently ignore it (the client would think it worked) and never pass it into ORDER BY.

Q6. Why can't you just parameterize the sort column to block injection? TRICKY
Placeholders bind values, not identifiers like column names or ASC/DESC. Sort columns must be mapped from an allow list to known column names in code.

Q7. Product wants users to filter on any of 30 attributes. How? TRICKY
Index the few that real traffic uses and allow-list those. For free-form combinations across many attributes, index the catalog in a search engine, which handles arbitrary filters and facets well, and keep the database API narrow.

Next: SD-C-009 · Idempotency Keys backendroadmap.com